Programmation des Systèmes Embarqués

Rapport de projet :

Navigation et Gestion Temporelle d'un Robot Autonome dans un Environnement Simulé pour Systèmes Embarqués

---

Réalisé par :
Theo Payet
Mathis Sommacal
Astrid RIVERA

Professeur :
Sellami, Bassem

Année universitaire 2024/2025

Table des matières

1 Introduction **2**

2 Modélisation de l'Environnement **2**
2.1 Configuration de l'Environnement 2
2.2 Gestion Thermique pour les systèmes embarqués 3
2.3 Zones de Température et de Refroidissement 4
2.4 Calcul des Récompenses et des Pénalités 4
2.5 Résumé de la Fonction de Récompense 5
2.6 Visualisation de l'Environnement 5

3 Importation des bibliothèques et initialisation **5**

4 Classe ThermalNavigationEnv **5**
4.1 Initialisation 6
4.2 Carte thermique 6
4.3 Gestion des étapes (`step`) 7

5 Algorithmes Implémentés **8**
5.1 Q-Learning 8
5.1.1 Description de l'algorithme 8
5.1.2 Logique d'apprentissage appliquée à l'environnement 9
5.1.3 Implémentation du Code 9
5.1.4 Adaptation au problème 10
5.1.5 Hyperparamètres utilisés 10
5.1.6 Résultats observés 10
5.2 SARSA 11
5.3 Deep Q-Network (DQN) 13

6 Simulation et Visualisation **14**
6.1 Rendu graphique 14
6.2 Courbes d'analyse 14

7 Analyse des Performances **15**
7.1 Récompenses Cumulées 15
7.2 Évolution de la Température 17
7.3 exploration et exploitation 21
7.4 Comparaison 22

8 Étude de Faisabilité Embarquée **22**

9 Conclusion et améliorations **23**

1 Introduction

Ce projet implémente un environnement personnalisé de navigation thermique à l'aide de la bibliothèque `Gymnasium`. L'objectif est de simuler un agent évoluant dans une grille avec des contraintes de température et des obstacles, tout en maximisant des récompenses.

Objectifs

- Créer un environnement *Gym* personnalisé représentant une grille avec des températures et obstacles.
- Implémenter trois algorithmes : **Q-Learning**, **SARSA** et **DQN**.
- Comparer les performances en termes de récompenses cumulées et de gestion thermique.
- Analyser les contraintes et proposer des améliorations pour une implémentation embarquée.

La navigation autonome et la gestion thermique sont des problématiques essentielles dans les systèmes embarqués modernes. Ces systèmes, comme les robots industriels, les drones ou les véhicules autonomes, opèrent souvent dans des environnements contraints où la température peut avoir un impact significatif sur leurs performances et leur durée de vie.

La gestion thermique est critique pour plusieurs raisons :

- **Fiabilité et Durabilité :** Une température excessive peut entraîner des défaillances matérielles ou réduire la durée de vie des composants électroniques.
- **Efficacité Énergétique :** Les mécanismes de refroidissement consomment de l'énergie, et leur activation doit être optimisée pour minimiser l'impact sur les ressources.
- **Sécurité :** Dans des environnements industriels ou des situations critiques (par exemple, un drone en vol), une mauvaise gestion thermique peut provoquer des accidents.

2 Modélisation de l'Environnement

Dans ce projet, un environnement personnalisé a été conçu à l'aide de la bibliothèque *Gym* pour simuler la navigation autonome d'un agent sous contraintes thermiques. Cet environnement est structuré comme une grille où chaque cellule représente une zone avec des propriétés spécifiques (température, obstacles, etc.).

2.1 Configuration de l'Environnement

L'environnement est une grille bidimensionnelle de dimensions 10×10, où chaque cellule a une température ambiante générée aléatoirement entre $20°C$ et $50°C$. La configuration spécifique est décrite ci-dessous :

- **Position de départ :** L'agent commence toujours en haut à gauche de la grille, à la position $(0, 0)$.
- **Objectif :** L'agent doit atteindre la position en bas à droite $(9, 9)$, marquée comme la zone de destination.

— **Obstacles :** Une cellule spéciale, située en $(4, 4)$, représente un « trou noir ». Si l'agent atteint cette cellule, il reçoit une forte pénalité et l'épisode se termine immédiatement.

— **Espaces d'action et d'observation :**

  — **Actions disponibles :** L'agent peut se déplacer vers le haut, le bas, la gauche ou la droite.

  — **Observations :** L'agent reçoit une matrice représentant la carte de température actuelle ainsi que sa position.

2.2 Gestion Thermique pour les systèmes embarqués

Mesure de la Dissipation de la Chaleur :
Pour gérer la dissipation de la chaleur, il est essentiel de mesurer la température et la consommation d'énergie des composants du système d'alimentation. Divers capteurs et outils peuvent être utilisés pour cette tâche :

— **Mesure de la température :** Les thermocouples, thermistances et caméras infrarouges peuvent mesurer la température des batteries, moteurs ou fils.

— **Mesure de l'énergie :** Les multimètres, oscilloscopes ou analyseurs de puissance sont utilisés pour mesurer la tension, le courant et la puissance du système.

— **Outils logiciels :** MATLAB et Simulink peuvent simuler et analyser la dissipation thermique du système électrique.

Réduction de la Production de Chaleur :
Pour réduire la génération de chaleur, il est crucial d'optimiser la conception et le fonctionnement du système d'alimentation :

— Utilisation de fils à faible résistance, de moteurs à haut rendement et de contrôleurs intelligents pour minimiser les pertes électriques.

— Adoption de niveaux de tension et de courant appropriés pour éviter les surcharges et les courts-circuits.

— Mise en œuvre de mécanismes de rétroaction et de protection pour prévenir une production excessive de chaleur.

Amélioration du Transfert de Chaleur :
Pour améliorer le transfert de chaleur, des matériaux et dispositifs thermiques peuvent être utilisés :

— Application de pâte thermique, tampons ou graisse pour augmenter la surface de contact entre les composants et le dissipateur thermique.

— Utilisation de ventilateurs, pompes ou tuyaux pour faire circuler l'air ou le liquide de refroidissement.

— Utilisation d'ailettes, radiateurs ou caloducs pour augmenter la surface d'émission de chaleur.

Contrôle de la Dissipation de la Chaleur :
Le contrôle de la dissipation thermique peut être réalisé par l'utilisation de capteurs, actionneurs et algorithmes :

— Capteurs de température pour détecter la température des composants et déclencher des dispositifs de refroidissement.

— Capteurs de puissance pour ajuster la tension, le courant ou la fréquence en fonction de la consommation d'énergie.

Test de la Dissipation Thermique :
Tester la dissipation thermique dans différentes conditions est essentiel pour évaluer et optimiser les performances :

— Utilisation de chambres thermiques, bancs de charge ou bancs d'essai pour appliquer divers paramètres et mesurer les résultats.

— Utilisation de MATLAB ou Simulink pour modéliser, simuler et optimiser les paramètres de conception et de fonctionnement.

2.3 Zones de Température et de Refroidissement

Chaque cellule de la grille a une température définie par une distribution uniforme entre $20°C$ et $50°C$. La température interne de l'agent est influencée par la température ambiante de la cellule où il se trouve. Cette température interne est initialisée aléatoirement entre $20°C$ et $40°C$ et évolue selon la relation :

$$T_{\text{interne}} \leftarrow T_{\text{interne}} + 0.1 \cdot (T_{\text{zone}} - T_{\text{interne}})$$

Si la température interne dépasse une valeur critique de $100°C$, l'agent est pénalisé.

2.4 Calcul des Récompenses et des Pénalités

La fonction de récompense est définie pour encourager l'agent à atteindre l'objectif tout en maintenant une température interne acceptable. Elle comprend des éléments de récompenses et de pénalités :

— **Récompenses positives :**

  — $+500$: Bonus accordé lorsque l'agent atteint la zone d'objectif $(9, 9)$.

  — **Récompense de proximité :** L'agent reçoit un bonus basé sur sa proximité avec l'objectif, défini comme :

$$R_{\text{proximité}} = \frac{10}{1 + \text{distance_objectif}}$$

  où la distance est calculée en norme Euclidienne.

— **Pénalités :**

  — **Pénalité thermique :** Si la température de la cellule où se trouve l'agent s'écarte de la température idéale ($37°C$), une pénalité proportionnelle est appliquée :

$$P_{\text{thermique}} = -0.1 \cdot |T_{\text{zone}} - 37|$$

  — **Surchauffe :** Si la température interne dépasse $100°C$, une pénalité additionnelle est appliquée pour chaque degré supplémentaire.

— **Trou noir :** Si l'agent atteint la cellule $(4, 4)$, une pénalité immédiate de -100 est appliquée, et l'épisode se termine.
— **Pénalité temporelle :** Une petite pénalité fixe (-0.1) est ajoutée pour chaque étape afin d'encourager des trajets courts.

2.5 Résumé de la Fonction de Récompense

La récompense totale à chaque étape est calculée comme suit :

$$R_{\text{total}} = R_{\text{objectif}} + R_{\text{proximité}} + P_{\text{thermique}} + P_{\text{surchauffe}} + P_{\text{trou_noir}} + P_{\text{temps}}$$

2.6 Visualisation de l'Environnement

Chaque cellule est visualisée avec un code couleur :

— **Rouge :** Températures élevées ($\sim 50°C$).
— **Bleu :** Températures basses ($\sim 20°C$).
— **Vert :** Position de l'objectif.

Le trou noir est représenté graphiquement avec une icône spécifique, et un effet sonore est joué lorsqu'il est atteint. La température interne de l'agent est tracée à chaque étape pour analyser sa gestion thermique.

3 Importation des bibliothèques et initialisation

Le projet est organisé en plusieurs parties :

— **Importation des bibliothèques et initialisation.**
— **Classe ThermalNavigationEnv** : création et gestion de l'environnement.
— **Simulation de l'agent** : gestion des actions et des récompenses.
— **Visualisation** : rendu graphique avec Pygame et graphiques d'analyse.
— **Exécution principale** : boucle de test de l'environnement.

Voici les bibliothèques utilisées :

— `numpy` : manipulation de tableaux et génération de cartes thermiques.
— `gymnasium` : création d'un environnement compatible avec les agents RL.
— `pygame` : visualisation de l'environnement et gestion des images/sons.
— `matplotlib` : traçage des courbes d'évolution des performances.

4 Classe ThermalNavigationEnv

La classe `ThermalNavigationEnv` est au cœur du projet et définit l'environnement personnalisé.

4.1 Initialisation

— Définition de la grille (taille et températures).

— Chargement des images (agent et obstacles) et des sons.

— Définition des espaces d'action et d'observation pour l'agent.

Code :

```python
class ThermalNavigationEnv(gym.Env):
    def __init__(self, grid_size=10, max_temp=100):
        super(ThermalNavigationEnv, self).__init__()

        pygame.mixer.init()
        self.agent_image = pygame.image.load("robot_head.png")
        self.agent_image = pygame.transform.scale(self.agent_image, (40, 40))

        self.hole_image = pygame.image.load("black_hole.png")
        self.hole_image = pygame.transform.scale(self.hole_image, (50, 50))
        self.hole_sound = pygame.mixer.Sound("bang.mp3")
        self.goal_sound = pygame.mixer.Sound("goal.mp3")

        self.grid_size = grid_size
        self.max_temp = max_temp
        self.temperature_map = self._generate_temperature_map()

        self.action_space = spaces.Discrete(4)
        self.observation_space = spaces.Box(
            low=0, high=max_temp,
            shape=(grid_size, grid_size, 2),
            dtype=np.float32
        )

        self.agent_position = [0, 0]
        self.goal_position = [grid_size - 1, grid_size - 1]
```

Listing 1 – Initialisation de l'environnement

4.2 Carte thermique

La carte thermique est générée de manière aléatoire pour chaque simulation.Elle permet de créer une map constituée de cellules de couleur qui symbolise les différentes température.

```python
def _generate_temperature_map(self):
    np.random.seed(42)
    return np.random.uniform(20, 50, (self.grid_size, self.grid_size))
```

Listing 2 – Carte thermique

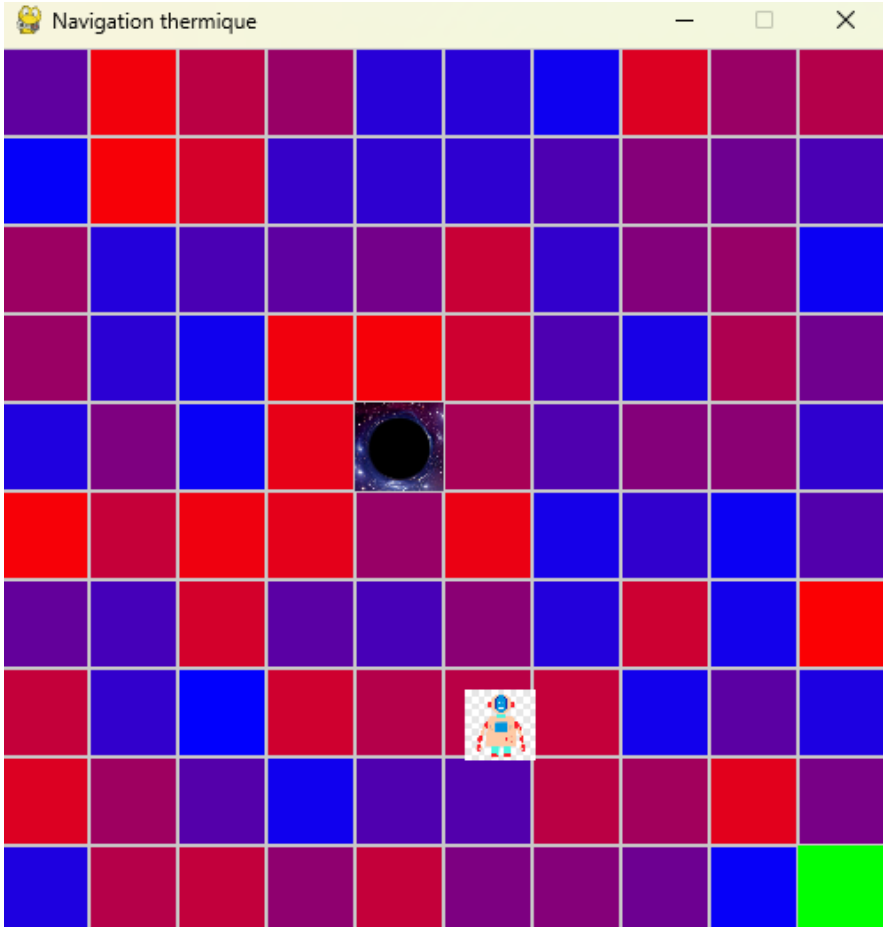


FIGURE 1 – carte thermique avec gradient de température

4.3 Gestion des étapes (`step`)

Cette méthode effectue les actions suivantes :

- Déplacement de l'agent.
  L'agent se déplacera selon 4 actions :
  - 0 pour Haut
  - 1 pour bas
  - 2 pour gauche
  - 3 pour droite
- Mise à jour de la température interne de l'agent.
  On calculera la differnce entre la température de l'agent et celle ambiante.Et on met multiplie par un facteur (ici ca sera 0.1) pour indiquer que l'agent absorbe une partie de la chaleur.

— Calcul de la récompense et vérification de la condition de fin. Le goal est marqué par un carré vert

```python
def step(self, action):
    if action == 0:  # Haut
        self.agent_position[0] = max(0, self.agent_position[0] - 1)
    elif action == 1:  # Bas
        self.agent_position[0] = min(self.grid_size - 1, self.agent_position[0] + 1)
    elif action == 2:  # Gauche
        self.agent_position[1] = max(0, self.agent_position[1] - 1)
    elif action == 3:  # Droite
        self.agent_position[1] = min(self.grid_size - 1, self.agent_position[1] + 1)

    zone_temp = self.temperature_map[self.agent_position[0], self.agent_position[1]]
    self.current_temperature += (zone_temp - self.current_temperature) * 0.1
    done = self.agent_position == self.goal_position
    reward = 100 if done else -abs(zone_temp - 37)
    return self._get_observation(), reward, done, {}
```

Listing 3 – Gestion des étapes (`step`)

5 Algorithmes Implémentés

Nous avons comparé trois algorithmes :

5.1 Q-Learning

Q-Learning est un algorithme d'apprentissage par renforcement hors-politique (*off-policy*) utilisé pour apprendre une politique optimale en fonction d'un environnement donné. Dans ce projet, il est utilisé pour guider un agent dans une grille 10×10 tout en respectant des contraintes thermiques et en évitant des obstacles.

5.1.1 Description de l'algorithme

L'algorithme Q-Learning met à jour une table $Q(s, a)$, où chaque paire état-action (s, a) est associée à une valeur. La règle de mise à jour est donnée par l'équation de Bellman :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

où :

— s et s' représentent respectivement l'état courant et l'état suivant.
— a est l'action prise dans l'état s, et a' est l'action optimale dans l'état s'.
— r est la récompense immédiate reçue après avoir exécuté a.
— α est le taux d'apprentissage (*learning rate*).
— γ est le facteur de discount (*discount factor*), qui pondère l'importance des récompenses futures.

5.1.2 Logique d'apprentissage appliquée à l'environnement

Dans cet environnement, Q-Learning suit les étapes suivantes :

1. L'agent commence dans l'état initial $(0, 0)$.
2. Une politique *epsilon-greedy* est utilisée pour choisir une action :
   — Avec une probabilité ϵ, une action aléatoire est choisie (**exploration**).
   — Avec une probabilité $1 - \epsilon$, l'action qui maximise la valeur $Q(s, a)$ est choisie (**exploitation**).
3. Après exécution de l'action, l'état suivant s' et la récompense r sont observés, et la valeur $Q(s, a)$ est mise à jour.
4. ϵ décroît progressivement après chaque épisode pour privilégier l'exploitation.

5.1.3 Implémentation du Code

L'implémentation de l'algorithme est décrite dans le code suivant :

```
def train_q_learning(env, episodes=500, alpha=0.1, gamma=0.99,
    epsilon=1.0, epsilon_min=0.01, epsilon_decay=0.995):
    q_table = np.zeros((env.grid_size, env.grid_size, env.
        action_space.n))  # Table Q initialis e    0
    rewards_history = []

    for episode in range(episodes):
        state = env.reset()
        state_pos = env.agent_position
        total_reward = 0
        done = False

        while not done:
            if random.uniform(0, 1) < epsilon:
                action = env.action_space.sample()  # Exploration
            else:
                action = np.argmax(q_table[state_pos])  #
                    Exploitation

            next_state, reward, done, _, _ = env.step(action)
            next_state_pos = env.agent_position

```

```
            row, col = int(state_pos[0]), int(state_pos[1])
            next_row, next_col = int(next_state_pos[0]), int(
               next_state_pos[1])

            q_table[row, col, action] = (1 - alpha) * q_table[row,
                col, action] + \
                alpha * (reward + gamma * np.max(q_table[next_row,
                    next_col, :]))

            state_pos = next_state_pos
            total_reward += reward

        epsilon = max(epsilon_min, epsilon * epsilon_decay)
        rewards_history.append(total_reward)

    return q_table, rewards_history
```

Listing 4 – Implémentation de Q-Learning

5.1.4 Adaptation au problème

L'algorithme Q-Learning est adapté à cet environnement en considérant :

- **Récompenses et pénalités :**
  - Récompense de $+200$ pour atteindre l'objectif $(9, 9)$.
  - Pénalités thermiques si la température interne dépasse $40°C$.
  - Pénalités importantes (-100) si l'agent atteint l'obstacle $(4, 4)$.
- **Exploration et exploitation :** La politique *epsilon-greedy* équilibre la découverte de nouvelles stratégies et l'utilisation des valeurs Q connues.

5.1.5 Hyperparamètres utilisés

Les hyperparamètres suivants ont été utilisés pour entraîner l'agent avec Q-Learning :

- Taux d'apprentissage (α) : 0.1
- Facteur de discount (γ) : 0.99
- ϵ initial : 1.0
- ϵ minimum : 0.01
- Décroissance de ϵ : 0.995
- Nombre d'épisodes : 500

5.1.6 Résultats observés

Après l'entraînement, les observations suivantes ont été faites :

- La table Q a convergé vers des valeurs stables, permettant à l'agent d'atteindre l'objectif efficacement.

— Les récompenses cumulées ont montré une amélioration progressive, indiquant un apprentissage réussi.
— La gestion thermique a été optimisée, et l'agent évite les obstacles critiques.

5.2 SARSA

SARSA utilise une politique en ligne pour la mise à jour :

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma Q(s',a') - Q(s,a) \right]$$

Résumé de l'algorithme SARSA (State-Action-Reward-State-Action)

SARSA (State-Action-Reward-State-Action) est un algorithme d'apprentissage par renforcement qui utilise une politique comportementale pour mettre à jour la fonction $Q(s,a)$. Contrairement au Q-learning, qui suit une approche hors politique (*off-policy*), SARSA est un algorithme en politique (*on-policy*), ce qui signifie qu'il apprend en suivant directement la politique d'exploration utilisée.
—

Formule de mise à jour

La fonction d'action-valeur $Q(s,a)$ est mise à jour selon la formule :

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma Q(s',a') - Q(s,a) \right]$$

où :

— s : État courant,
— a : Action choisie dans l'état s,
— r : Récompense observée après exécution de a,
— s' : Nouvel état atteint après a,
— a' : Action choisie dans l'état s' selon la politique actuelle,
— α : Taux d'apprentissage (*learning rate*),
— γ : Facteur d'actualisation (*discount factor*).
—

Étapes de l'algorithme SARSA

1. **Initialisation** :
   — Initialiser la fonction $Q(s,a)$ pour toutes les paires (s,a) (généralement à zéro ou à une petite valeur aléatoire).
2. **Interaction avec l'environnement** :
   — Initialiser l'état s.
   — Choisir une action a en suivant une politique π (par exemple, ϵ-greedy basée sur Q).

3. **Boucle principale** (jusqu'à la convergence ou un nombre d'épisodes prédéfini) :
   (a) Exécuter l'action a, observer la récompense r et le nouvel état s'.
   (b) Choisir une nouvelle action a' dans l'état s' en suivant la politique π.
   (c) Mettre à jour la valeur $Q(s, a)$ selon la formule de mise à jour :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma Q(s', a') - Q(s, a) \right].$$

   (d) Mettre à jour l'état et l'action : $s \leftarrow s'$, $a \leftarrow a'$.

—

Composants clés de SARSA

— **Politique en politique (*on-policy*)** : La politique utilisée pour explorer est également celle utilisée pour mettre à jour $Q(s, a)$.

— **Politique ϵ-greedy** : Équilibre entre exploration (choix aléatoire) et exploitation (choix basé sur Q).

— **Facteur d'actualisation** γ : Contrôle l'importance des récompenses futures.

— **Taux d'apprentissage** α : Détermine la vitesse de mise à jour de $Q(s, a)$.

—

Différences principales avec Q-learning

— SARSA est un algorithme en politique (*on-policy*), tandis que Q-learning est hors politique (*off-policy*).

— Dans SARSA, l'action a' utilisée dans la mise à jour de Q est choisie selon la même politique d'exploration.

— En pratique, SARSA peut être plus adapté pour des environnements où une exploration prudente est nécessaire.

—

Avantages et limites de SARSA

Avantages

— Plus stable dans des environnements stochastiques.

— L'apprentissage est aligné avec la politique suivie, ce qui peut être utile dans des scénarios où des actions risquées doivent être évitées.

Limites

— Peut converger plus lentement que Q-learning.

— Moins efficace dans des environnements où une exploration agressive est souhaitée.

5.3 Deep Q-Network (DQN)

— Une architecture neuronale approximant les Q-values.

— Stabilisation via *replay buffer* et réseau cible.

Les hyperparamètres utilisés pour tous les algorithmes incluent :

— Taux d'apprentissage $\alpha = 0.1$

— Facteur de discount $\gamma = 0.99$

— Décroissance d'exploration $\epsilon = 0.995$

DQN est une extension de l'algorithme de Q-learning qui utilise un réseau neuronal profond pour approximer la fonction $Q(s, a)$. Cet algorithme est particulièrement adapté pour des espaces d'états complexes ou continus.
L'objectif est d'apprendre une approximation de la fonction $Q(s, a)$, qui donne la récompense cumulative maximale possible pour chaque paire (s, a).

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Dans DQN, la fonction $Q(s, a)$ est approximée par un réseau neuronal $Q_\theta(s, a)$, où θ représente les poids du réseau.

—

Étapes de l'algorithme DQN

1. **Initialisation** :
   — Initialiser un réseau neuronal Q_θ pour approximer la fonction $Q(s, a)$.
   — Initialiser un réseau cible Q_{θ^-} (copie de Q_θ).
   — Initialiser un *replay buffer* pour stocker les transitions $(s, a, r, s', \text{done})$.
2. **Interaction avec l'environnement** :
   — Choisir une action a dans l'état s en utilisant une politique ϵ-greedy.
   — Exécuter l'action a, observer la récompense r et le nouvel état s'.
   — Stocker la transition $(s, a, r, s', \text{done})$ dans le replay buffer.
3. **Échantillonnage du replay buffer** :
   — Échantillonner un mini-lot de transitions $(s, a, r, s', \text{done})$ depuis le replay buffer.
4. **Calcul de la cible** :
$$y = \begin{cases} r + \gamma \max_{a'} Q_{\theta^-}(s', a') & \text{si } s' \text{ n'est pas terminal,} \\ r & \text{si } s' \text{ est terminal.} \end{cases}$$
5. **Mise à jour des poids du réseau** Q_θ : Minimiser l'erreur quadratique entre $Q_\theta(s, a)$ et y :
$$L(\theta) = \mathbb{E}\left[(y - Q_\theta(s, a))^2 \right].$$
6. **Mise à jour du réseau cible** :
$$Q_{\theta^-} \leftarrow Q_\theta \quad \text{(périodiquement).}$$

—

Composants clés de DQN

- **Réseau neuronal** : Approxime la fonction $Q(s, a)$.
- **Replay buffer** : Stocke les transitions $(s, a, r, s', \text{done})$ pour briser la corrélation entre les données consécutives.
- **Réseau cible (Q_{θ^-})** : Stabilise l'apprentissage en fixant temporairement les cibles.
- **Politique ϵ-greedy** : Équilibre exploration et exploitation.
-

Améliorations possibles de DQN

- **Double DQN** : Réduit le biais en séparant le calcul de l'action optimale et de sa valeur.
- **Dueling DQN** : Apprend à séparer la valeur de l'état et l'avantage des actions.
- **Prioritized Experience Replay** : Donne une priorité plus élevée aux transitions importantes.

6 Simulation et Visualisation

6.1 Rendu graphique

L'environnement est rendu en utilisant `Pygame` pour visualiser l'état de la grille, la position de l'agent et les obstacles.

```python
def render(self, mode='human'):
    self.screen.fill((255, 255, 255))
    for i in range(self.grid_size):
        for j in range(self.grid_size):
            temp = self.temperature_map[i, j]
            color = (int(255 * (temp - 20) / 30), 0, int(255 * (50 - temp) / 30))
            pygame.draw.rect(self.screen, color, (j * 50, i * 50, 50, 50))
    self.screen.blit(self.agent_image, (self.agent_position[1] * 50, self.agent_position[0] * 50))
    pygame.display.flip()
```

Listing 5 – Rendu graphique avec Pygame

6.2 Courbes d'analyse

Les performances de l'agent (température interne et récompenses cumulées) sont analysées à l'aide de graphiques.

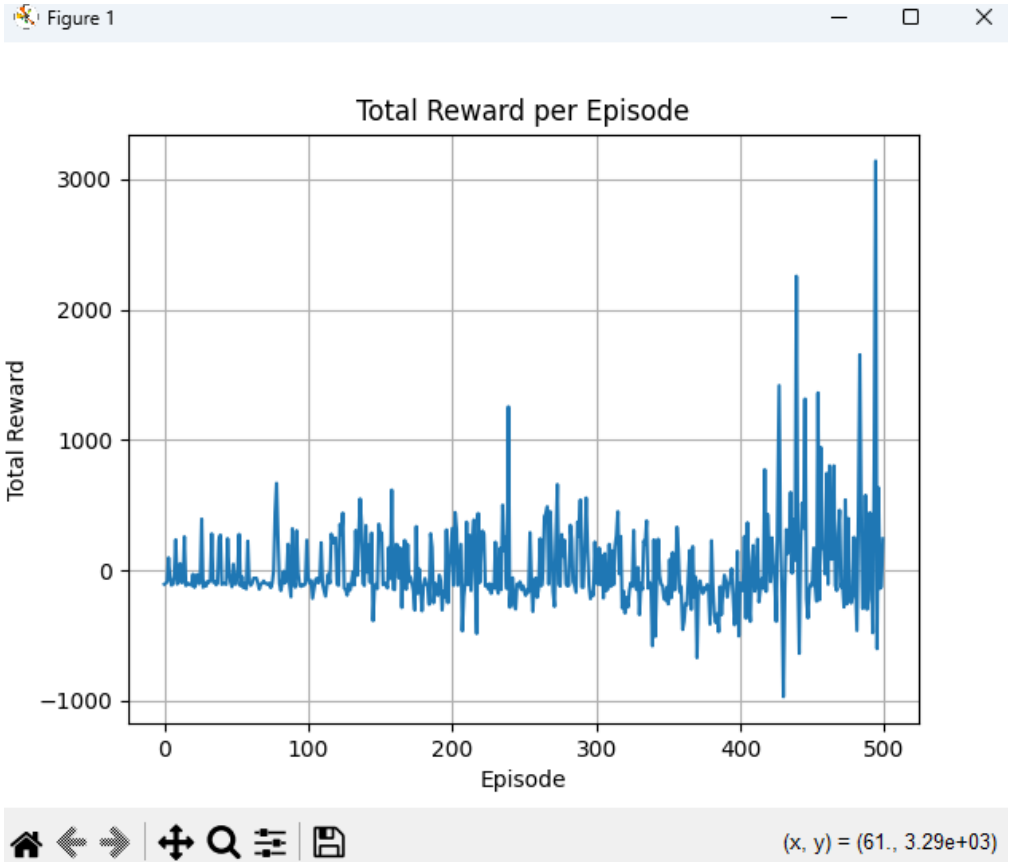
```python
def plot(self):
    plt.plot(self.agent_temp_history)
    plt.axhline(y=37, color='r', linestyle='--', label='
        Temp rature id ale')
    plt.xlabel('Temps')
    plt.ylabel('Temp rature')
    plt.title('Temp rature de l'agent au cours du temps')
    plt.legend()
    plt.show()
```

Listing 6 – Courbes d'analyse

7 Analyse des Performances

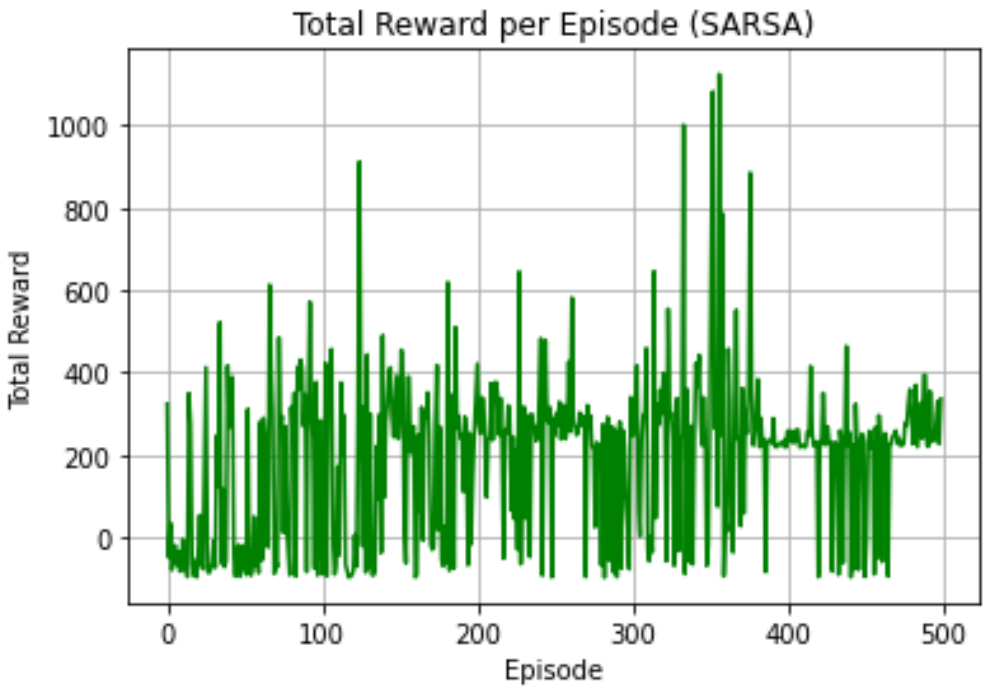
7.1 Récompenses Cumulées

Les performances de chaque algorithme ont été évaluées en termes de récompenses cumulées :



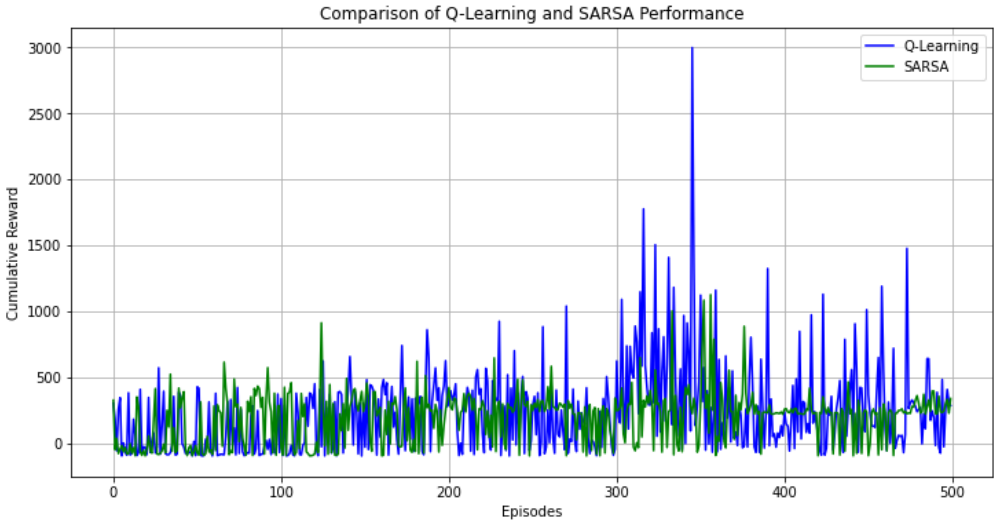
justification=centering

FIGURE 2 – récompense cumulés pour l'algorithme Q-learning



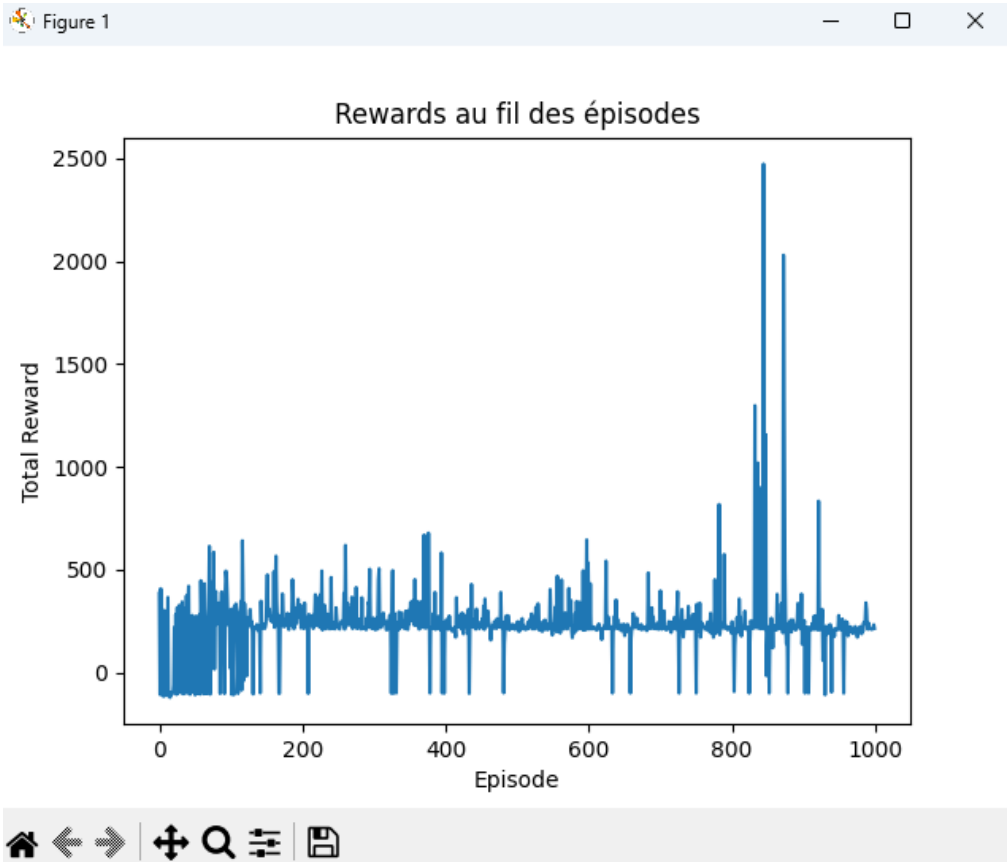
justification=centering

FIGURE 3 – récompense cumulés pour l'algorithme SARSA



justification=centering

FIGURE 4 – comparaison récompense cumulés pour l'algorithme SARSA et Q-learning
On remarque que les deux alogirthme (SARSA et QL ont quasi les même résultats donc on essay de voir si en implémentant un deep Q$_{l}earningonauneamlioration$



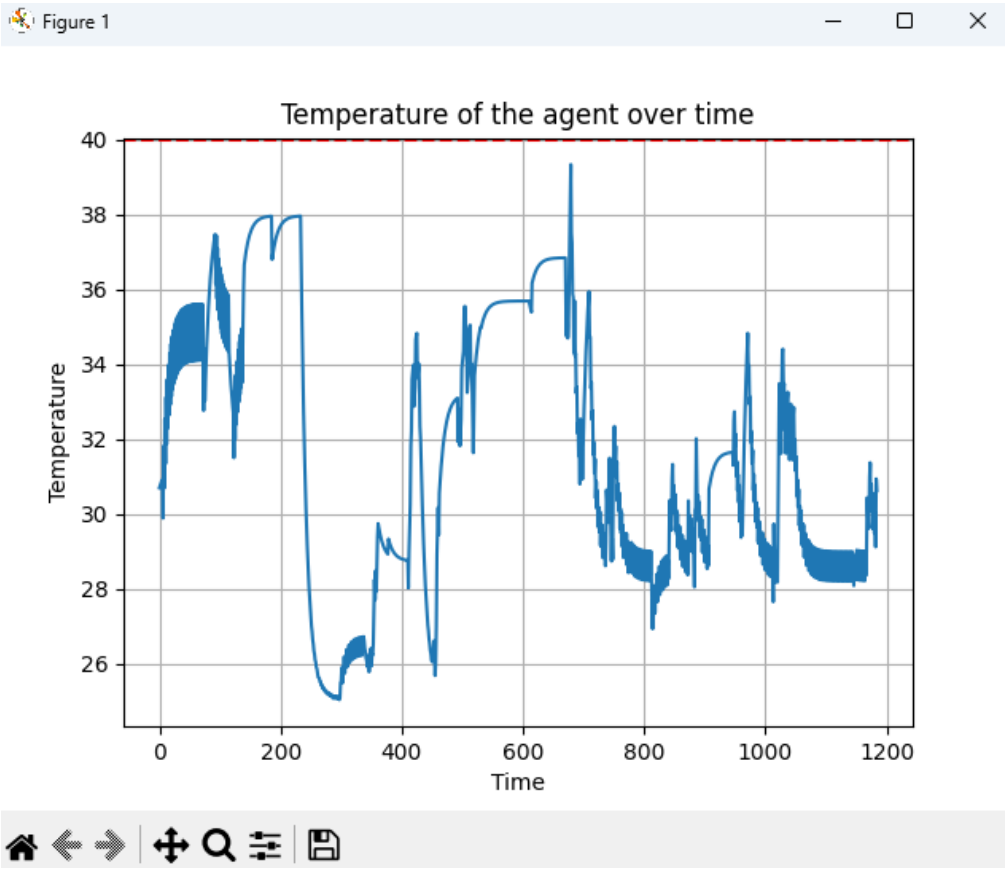
justification=centering

FIGURE 5 – récompense cumulées pour l'algorithme DQN

On remarque qu'au bout de 800 episode on a obtenue un grand de récompense, il y a beaucoup moins de pic négatif et plus de pic avec de grande récompense donc cette algorithme est nettement meilleur
Maintenant interressont nous au température.

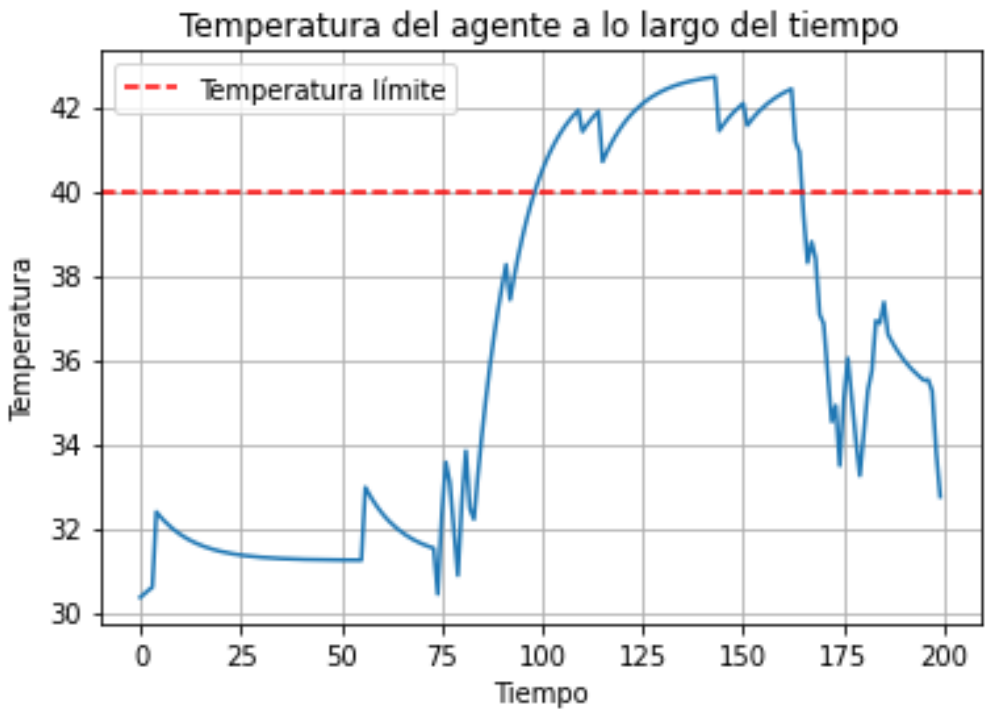
7.2 Évolution de la Température

La gestion thermique de l'agent a également été analysée, on a fixé un seuil a 40 degrès au delà de cette température il est dangereux pour le robot et ces composant de résister :



justification=centering

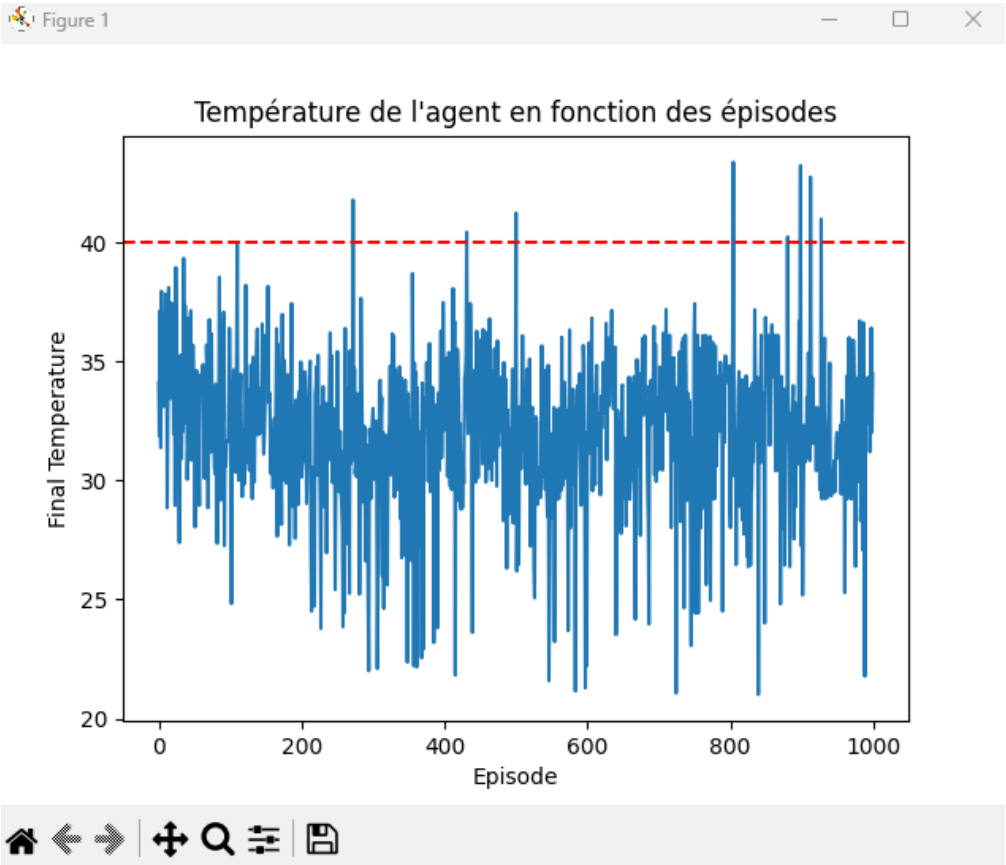
FIGURE 6 – Évolution de la température de l'agent pour l'algorithme Q-learning



justification=centering

FIGURE 7 – Évolution de la température de l'agent pour l'algorithme SARSA

En plus d'etre quasi égale en terme de récompesne cumulé qu'avec l'algorithme Q-learning il est plus chaud et dépasse même son seil critique de focntionnement donc il est clairement moin bon que l'algorithme Q-learning Regardons maintenant celui du DQN :

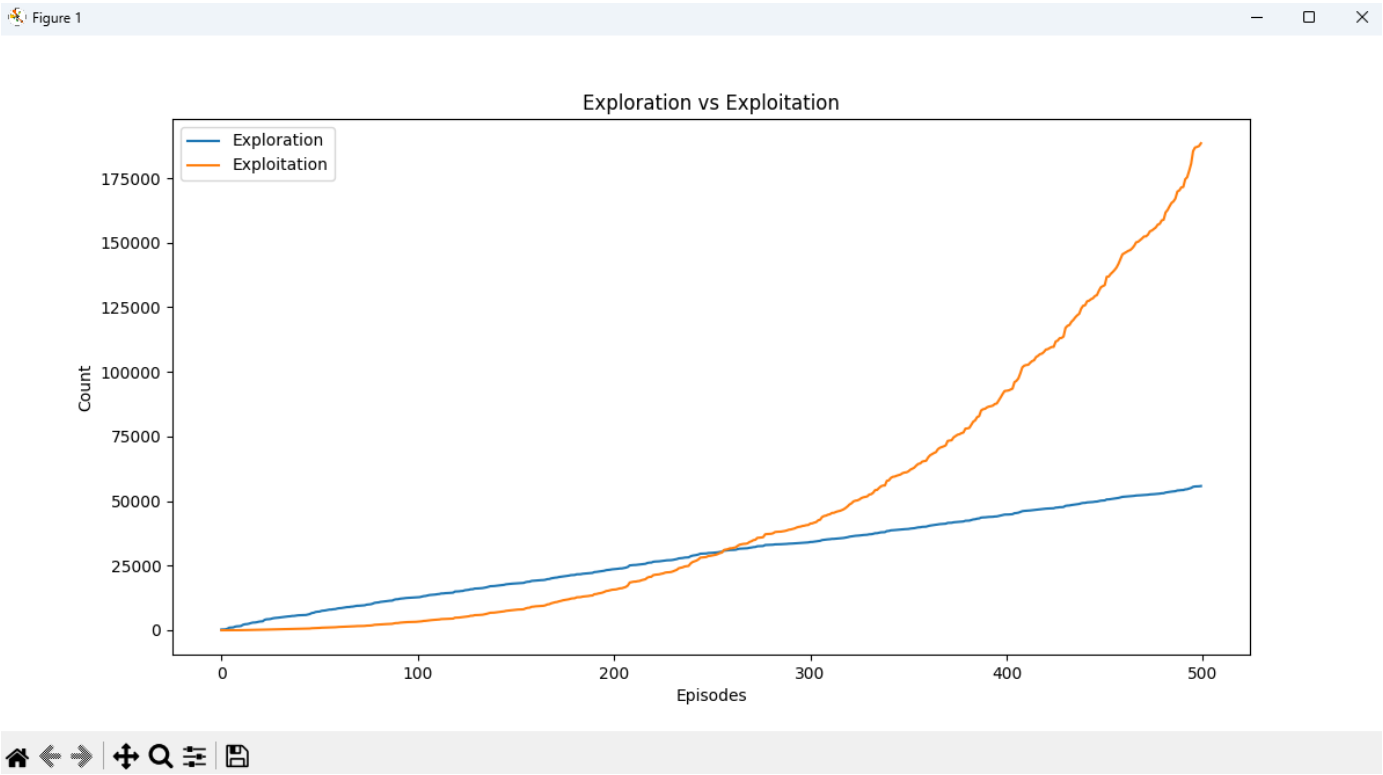


justification=centering

FIGURE 8 – Évolution de la température de l'agent pour l'algorithme DQN

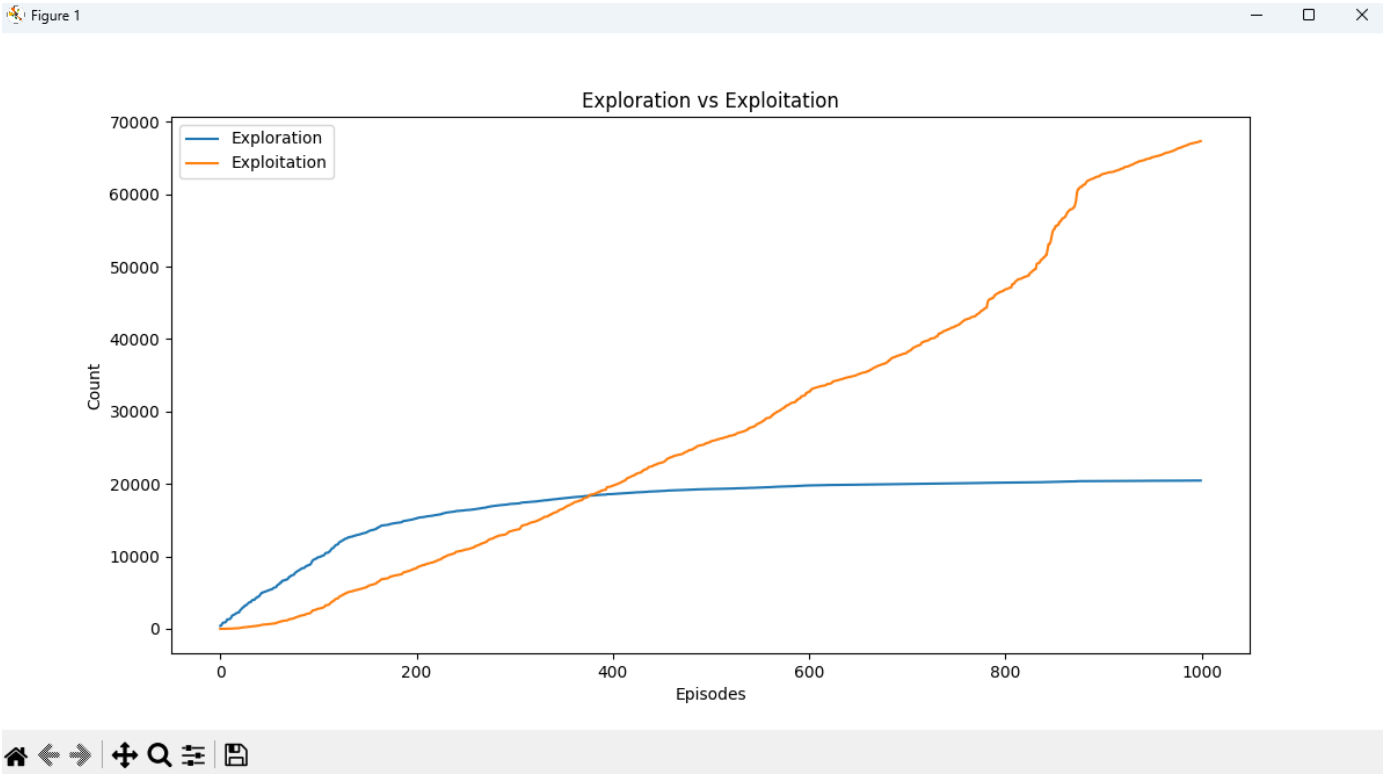
On remarque que pour l'alorgithme de q-learning sa tempéture est mieux optimiser il dépasse pas son seuil et reste en moyenne à 32°.En revanche l'algorithme de DQN lui ne permet pas au robot de se déplacer sans dépasser son seuil de température, à certains moment il y a des pic de surchauffe mais de manière général il reste en dessous de 40° avec un moyenne à 35° , ce qui beaucoup plus qu'avec l'algorithme de Q-learning.

7.3 exploration et exploitation



justification=centering

FIGURE 9 – Exploration et Exploittion en fonction des itération pour l'alogithme Q-learning



justification=centering

FIGURE 10 – Exploration et Exploittion en fonction des itération pour l'alogithme DQN

Remarque : les graphique ne sont pas mis en pourcentage ce qui signifie ici que ce sont la somme du nombre de fois que l'agent fait de l'exploration ou de l'exploitation.
Globalement on voit que le facteur ϵ_{greedy} a bien été choisi car en effet il y a au debut des épisode plus d'exploration et plus on progresse dans le processus et plus l'agent exploite.Ceci est représenté par la courbe d'exporation qui est au dessus de celle d'exploitation et au d'un certain nombre d'itération il stagne pour laisser place à l'exploitation.

7.4 Comparaison

- DQN a montré une convergence rapide et une meilleure gestion des récompenses.
- SARSA est plus stable dans des environnements dynamiques.
- Q-Learning, bien que performant, est sensible aux variations de politique.

8 Étude de Faisabilité Embarquée

- **Contraintes de calcul :** Les ressources nécessaires pour DQN (réseaux neuronaux) sont importantes comparées à SARSA et Q-Learning.
- **Mémoire :** La table $Q(s, a)$ pour Q-Learning et SARSA devient grande avec des environnements complexes.
- **Énergie :** DQN consomme plus d'énergie en raison des calculs de gradient.

Des optimisations possibles incluent :

- Réduction de la taille des réseaux neuronaux.
- Implémentation en C pour réduire la latence sur microcontrôleurs.
- Utilisation d'approximations basées sur les tables Q pour simplifier les calculs.

9 Conclusion et améliorations

Ce projet est une base solide pour expérimenter avec des agents RL dans un environnement personnalisé. Voici quelques pistes d'amélioration :

- Ajouter plus d'obstacles ou des zones spéciales (accélération, ralentissement).
- Tester différents algorithmes sur cet environnement.
- Rendre les cartes thermiques dynamique

En comparant tout les differents algorithmes celui DQn est meilleur mais il surchauffe l'appareil.